Documentación Técnica: Tarea 1-Analisis Sintactico

Desarrollador: Juan Camilo Lozano Cortés.

Lenguaje: Python.

1. Preparación del Entorno:

Se configuró el entorno de desarrollo en Kali Linux, instalando las dependencias necesarias (lark, pydot) y el motor de renderizado graphviz para la generación de los Árboles de Sintaxis Abstracta.

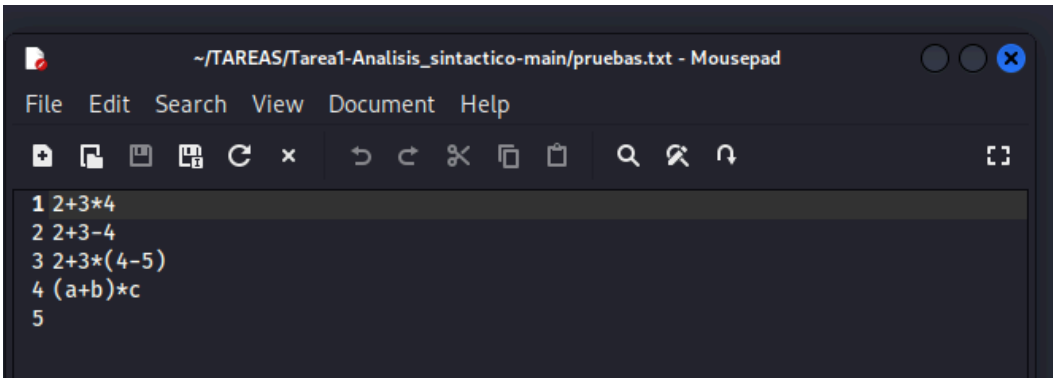
Se eligió Lark por su versatilidad para procesar gramáticas recursivas y su capacidad nativa para transformar derivaciones sintácticas en representaciones gráficas (AST), cumpliendo con los requisitos de eficiencia y claridad visual exigidos en la práctica."

2. Definición de la Gramática y Código:

```python
import sys
import os
from lark import Lark, tree

grammar = """
    ?start: e

    ?e: t
      | e opsuma t    -> suma

    ?t: f
      | t opmul f     -> multiplicacion

    ?f: id
      | num
      | "pari" e "pard"

    opsuma: "+" | "-"
    opmul: "*" | "/"
    id: /[a-zA-Z_][a-zA-Z0-9_]*/
    num: /\d+/

    %import common.WS
    %ignore WS
"""

def procesar_archivo(ruta_txt):
    if not os.path.exists(ruta_txt):
        print(f"Error: No se encuentra el archivo {ruta_txt}")
        return

    parser = Lark(grammar, start='start', parser='earley')

    with open(ruta_txt, 'r') as f:
        lineas = f.readlines()

    for i, linea in enumerate(lineas):
        cadena = linea.strip()
        if not cadena:
            continue

        try:
            # Reemplazo de parentesis por tokens de la gramatica
            proc = cadena.replace("(", "pari").replace(")", "pard")
            arbol = parser.parse(proc)

            nombre_salida = f"ast_linea_{i+1}.png"
            tree.pydot__tree_to_png(arbol, nombre_salida)
            print(f"Linea {i+1} OK: {cadena} -> {nombre_salida}")
        except Exception as e:
            print(f"Error en linea {i+1} ({cadena}): {e}")
```

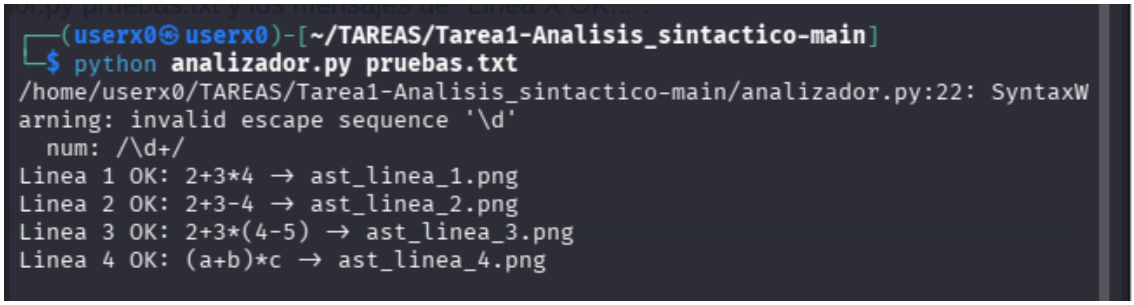
Se implementó una gramática independiente del contexto (GIC) que respeta la jerarquía de operaciones. Se utilizaron reglas recursivas para permitir expresiones de longitud arbitraria y el motor Earley para procesar la estructura sintáctica.

3. Archivo de Entrada (pruebas.txt)



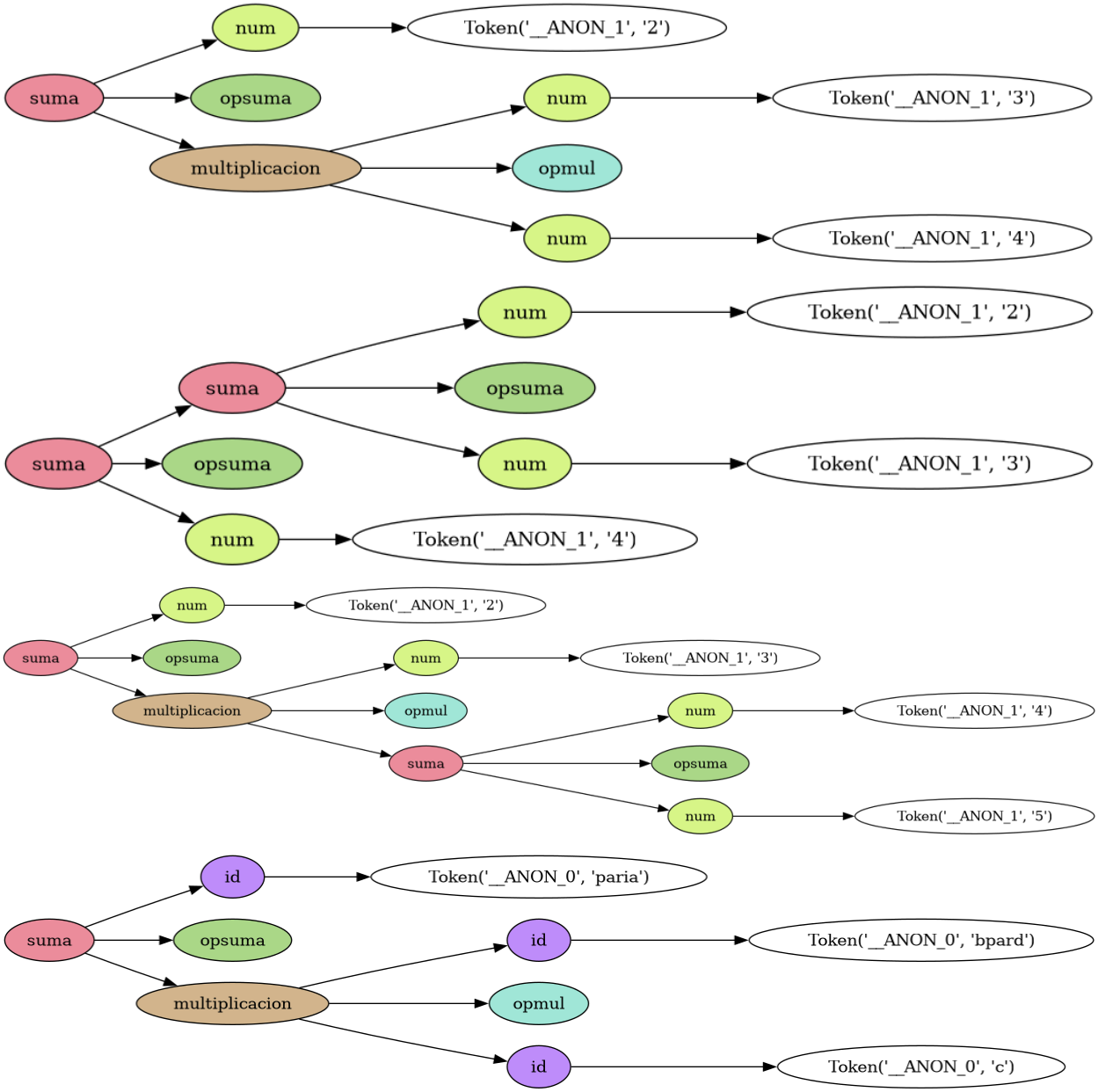
Se definieron diversos casos de prueba, incluyendo operaciones simples, uso de paréntesis y combinaciones de multiplicaciones con sumas para verificar que el parser respeta la precedencia matemática."

4. Ejecución del Programa:



Ejecución del analizador sobre el archivo de pruebas. El sistema procesa cada línea secuencialmente, genera el árbol jerárquico en memoria y lo exporta a un archivo de imagen PNG.

5. Análisis de Resultados (Los AST)



Aca podemos observar como al ejecutar el programa nos devuelve 4 archivos: “ast_linea_1.png”,“ast_linea_2.png”,“ast_linea_3.png”,“ast_linea_4.png”

En la Línea 1 (2+3*4), se observa cómo el nodo de la multiplicación queda en un nivel inferior al de la suma, forzando su evaluación prioritaria."

En la Línea 4 ((a+b)*c), los tokens pari y pard alteran la estructura natural, encapsulando la suma como un bloque primario antes de la multiplicación."